

Mock School System - Attacks and Countermeasures Life Cycle

Cross Site Scripting (XSS) Attack

Research Assistant & Author Brian Alexis Almaguer
Principle Investigator Dr. Jun Dai

Background

You have access to and manage a school system and website to track students, teachers, and grades. This web based application hosts information that is considered Personally Identifying Information (PII) that can be accessed by a one-to-one (1:1) relationship, respectively, by the owner of the information; Teachers and Administrators, however, have a one-to-many (1:*) relationship to the data since they manage the records and there updates. The Web application is a three-layer application; it is coded in PHP, severd to the web using Apache, and uses MySQL to store the user data. The site has a log-in that creates one of two 1:1 or 1:* views for the data. The 1:* view allows for the data to be edited in an editing view. All of this is designed and needed by the school whose systems you manage.

Scenario

A new bug within the school web application has been discovered in which the website data can be altered to allow a user to change their account creation payload. This is a common Cross Site Scripting (XSS) attack; The question is if the XSS attack is caused by the web application, the software running the web application, or the methods programmed.

SSDLC

The Secure Software Development Life Cycle (SSDLC), is what we are going to follow to since it helps organizations develop secure software. SSDLC is also what we are going to follow, since software, made with SSDLC, is developed with security considerations at every stage of the development process. More can be read here¹ in the wonderful Hackerone article.

Planning

1. Step 1 - Recreate the bug or attack

In the planning phase of the SSDLC, since we already have a preexisting product, we are going to need to recreate the bug or, in this case, the XSS attack. The following is the HTML code that is being edited and that has allowed a user to create an account.

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <title>Signup Form</title>
6  </head>
7
8  <body>
9    <h2>Signup Form</h2>
10   <form method="post" action="signup.php">
11     <label for="userID">User ID:</label>
12     <input type="text" id="userID" name="userID" required><br>
13
14     <label for="userPW">Password:</label>
15     <input type="password" id="userPW" name="userPW" required><br>
16
17     <label for="firstname">First Name:</label>
```

¹<https://www.hackerone.com/knowledge-center/what-ssdlc-secure-software-development-life-cycle>

```

18     <input type="text" id="firstname" name="firstname" required><br>
19
20     <label for="lastname">Last Name:</label>
21     <input type="text" id="lastname" name="lastname" required><br>
22
23     <label for="address">Address:</label>
24     <input type="text" id="address" name="address" required><br>
25
26     <label for="city">City:</label>
27     <input type="text" id="city" name="city" required><br>
28
29     <label for="state">State:</label>
30     <input type="text" id="state" name="state" required><br>
31
32     <label for="zip">Zip Code:</label>
33     <input type="text" id="zip" name="zip" required><br>
34
35     <label for="phone">Phone Number:</label>
36     <input type="text" id="phone" name="phone" required><br>
37
38     <input type="hidden" id="userLevel" name="userLevel" value="student">
39
40     <input type="submit" value="Submit">
41 </form>
42 </body>
43
44 </html>

```

In this code there is one input tag that has allowed users to change there account level; below is that tag of interest.

```

1 <input name="mylevel" type="hidden" id="mylevel" value="student">

```

To recreate the attack, all one needs to do is to change the value in HTML in inspect element on a web browser.

2. Step 2 - Risk Assessment

Not all attacks/bugs actually need a Risk Assessment, since a large majority of them have been categorized on site such as OWASP² (Open Worldwide Application Security Project). In this case, it is really clear what needs to be done on the web application.

3. Step 3 - Mitigation

Since the Attack Recreation and Risk Assessment defined exactly what we are dealing with, we now need to research and catalog what is out there for mitigation's. A great starting point is to look into articles OWASP has exactly like the following³.

Requirements and Analysis

1. Step 1 - Analysis

Once the research has been completed on the attack and its recreation, the underlying technology or code needs to be connected to your work. Doing this makes the next step easier to do.

2. Step 2 - Requirements

This step can be fleshed out until the connection between the attack/bug is connected to the technology or code. That connection essentially writes your requirements out for you; it is your job to expand on those connections into simpler explanations and ideas.

For our specific example project, our mistake stems from a hidden value set on the client side that determines how the website creates a user account. This value needs to be secured and/or removed as a requirement.

Design and Prototyping

1. Step 1 - Design

Not all designs need to be implemented in the product to work during this phase. Some designs can be

²<https://owasp.org/about/>

³<https://owasp.org/www-community/attacks/xss/>

generated at a high level without actually writing code. In our case, we don't need to change any code to make our design; as an example, let us say the design is to remove the HTML tag completely since our analysis points to the hidden value being client-side. This means we need to refactor our code to move the data to the server side. Below is an example of a design that does use code

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <title>Signup Form</title>
6  </head>
7
8  <body>
9    <h2>Signup Form</h2>
10   <form method="post" action="signup.php">
11     <label for="userID">User ID:</label>
12     <input type="text" id="userID" name="userID" required><br>
13
14     <label for="userPW">Password:</label>
15     <input type="password" id="userPW" name="userPW" required><br>
16
17     <label for="firstname">First Name:</label>
18     <input type="text" id="firstname" name="firstname" required><br>
19
20     <label for="lastname">Last Name:</label>
21     <input type="text" id="lastname" name="lastname" required><br>
22
23     <label for="address">Address:</label>
24     <input type="text" id="address" name="address" required><br>
25
26     <label for="city">City:</label>
27     <input type="text" id="city" name="city" required><br>
28
29     <label for="state">State:</label>
30     <input type="text" id="state" name="state" required><br>
31
32     <label for="zip">Zip Code:</label>
33     <input type="text" id="zip" name="zip" required><br>
34
35     <label for="phone">Phone Number:</label>
36     <input type="text" id="phone" name="phone" required><br>
37
38     <input type="submit" value="Submit">
39   </form>
40 </body>
41
42 </html>
```

2. Step 2 - Prototyping

Like Design, prototyping does not need to be implemented in the product; prototypes can be made outside of the larger product to show proof of designs in that way the product does not receive any unguided development changes.

As an example prototype, below is an SQL statement and SQL Schema that inserts into our authentication table

SQL Schema

```
1  CREATE TABLE `Authenticate` (
2    `userID` varchar(100) DEFAULT NULL,
3    `userPW` varchar(255) DEFAULT NULL,
4    `userLevel` varchar(100) NOT NULL,
5    `U_ID` int(10) NOT NULL
6  ) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4;
```

SQL Prototype

```
1  // Insert into Authenticate table
2  $sql_auth = "INSERT INTO Authenticate (userID, userPW, userLevel) VALUES ('$userID', '
    $userPW', 'student')";
```

Please note that our prototype example here is a naive solution since we just hard coded out value into our SQL statement.

Development

1. Step 1 - Version Control or Backups
In Development, the very first thing you do is back up or version control your original program and settings before testing your prototypes on the product. Data should also be backed up just in case of a full system breakdown.
2. Step 2 - Run/Create a Non-Production Environment
Part of development is testing; true product testing can be done when you have a mirrored live or local product. In our case, we have a code ready to be edited live to see the change made live.
3. Step 3 - Integration
This step involves or effort, as you now have to integrate your design and prototype into the mock product before attempting a simulated attack. This step can happen multiple times over multiple days if we are going to make a countermeasure for a larger or more complicated attack.

Deployment

Deployment is the most important step as a maintainer since your web application will have to be down preventing any of your uses from accessing the data they need. Depending on the strategy used in the organization, a change process would be needed before making any changes. In the end, the process always follows an unfreeze-change-freeze flow.

Maintenance

Common regular web application maintenance should include performing daily backups of both application code and database content, monitoring server logs for errors or suspicious activities, and periodically updating the security of all dependencies and frameworks. In this scenario, it is imperative that backups of the code are saved.